

Examen – Reservas de Hotel



Rúbrica automática (nota por ejercicio)

La nota se calcula sobre **8 puntos** (y se convierte a **/10**).

Ejercicio	Puntos	Archivo(s) evaluados	Tests
Ejercicio1 Dominio	1.5	<code>src/domain/models.ts</code>	<code>tests/domainModels.test.ts</code>
Ejercicio2 Lógica pura	1.5	<code>src/domain/logic.ts</code>	<code>tests/logic.test.ts</code> (si existe) o <code>tests/reservas.test.ts</code> (según proyecto)
Ejercicio3 SQLite	1.5	<code>src/repositories/SQLiteReservaRepository.ts</code>	<code>tests/repositorioReservasHotelSqlite.test.ts</code>
Ejercicio4 REST	1.5	<code>src/rest/ReservaRestService.ts</code>	<code>tests/apiReservasHotel.test.ts</code>
Ejercicio5 Servicio	1.0	<code>src/services/ReservaService.ts</code>	<code>tests/ServicioReservasHotel.test.ts</code> (parte servicio)
Ejercicio6 Sync	1.0	<code>src/services/sync.ts</code>	<code>tests/ServicioReservasHotel.test.ts</code> (parte sync)

Regla de puntuación dentro de cada ejercicio:

- Si pasan **todos** los tests del ejercicio → obtienes el **100%** de sus puntos.
- Si fallan algunos → puntos **proporcionales**: `puntos * (tests_pasados / tests_totales_del_ejercicio)`.

Ejercicio1 – Dominio (1,5)

 `src/domain/models.ts`

Define:

`enum EstadoReserva`

Valores: `"CONFIRMADA"` | `"CANCELADA"` | `"FINALIZADA"`

`interface Cliente`

- `id: string`
- `nombre: string`
- `email: string`

`interface Habitacion`

- `id: string`
- `numero: string`
- `tipo: string`
- `precioPorNoche: number`

`interface Reserva`

- `id: string`
- `clienteId: string`
- `habitacionId: string`
- `fechaEntrada: string` (ISO yyyy-mm-dd)
- `fechaSalida: string` (ISO yyyy-mm-dd)
- `estado: EstadoReserva`

Ejercicio2 – Lógica pura (1,5)

 `src/domain/logic.ts`

`calcularNoches(fechaEntrada: string, fechaSalida: string): number`

- Devuelve el número de noches (días) entre las dos fechas.
- Las fechas deben estar en formato **ISO de fecha**: "YYYY-MM-DD"

Ejemplo válido: "2026-01-13"

- Lanza un **Error** si:
 - alguna de las fechas no es válida.
 - `fechaSalida` es **igual o anterior** a `fechaEntrada`.

`calcularPrecioTotal(reserva: Reserva, habitacion: Habitacion): number`

- Calcula el precio total de la reserva.
- Debe devolver:
 - `calcularNoches(reserva.fechaEntrada, reserva.fechaSalida) * habitacion.precioPorNoche`
- Si `calcularNoches` lanza error, este error debe propagarse.

`hayConflicto(a: Reserva, b: Reserva): boolean`

- Devuelve **false** si las reservas pertenecen a **habitaciones distintas**.
- Devuelve **false** si **alguna** de las reservas está en estado **CANCELADA**.
- En caso contrario, existe conflicto si las fechas se solapan según un **intervalo semiabierto** [`entrada`, `salida`):
 - Hay conflicto si `max(entradaA, entradaB) < min(salidaA, salidaB)`
- Si una reserva empieza exactamente cuando la otra termina, **no hay conflicto**.

Ejercicio 3 – SQLite (1,5)

 **Archivo:** `src/repositories/SQLiteReservaRepository.ts`

Debes implementar un **repositorio SQLite** para persistir reservas.
Este repositorio se usa desde los tests para comprobar CRUD y sincronización.

Métodos a implementar

`init(): Promise<void>`

- Inicializa la conexión a la base de datos.
- Crea la tabla **reservas** **si no existe**.
- Debe ser **idempotente** (si se llama más de una vez, no debe fallar).

Recomendación: definir **id** como clave primaria para poder evitar duplicados.

`create(reserva: Reserva): Promise<void>`

- Inserta una reserva en la tabla.
- Debe guardar todos los campos de la reserva (incluido **estado**).
- Si el **id** ya existe, puede lanzar error (dependiendo del diseño), pero **no debe duplicar**.

`findAll(): Promise<Reserva[]>`

- Devuelve **todas** las reservas almacenadas.
 - Orden recomendado: por `fechaEntrada` ascendente (si no se especifica otro en los tests).
-

`findById(id: string): Promise<Reserva | null>`

- Devuelve la reserva con ese `id`.
 - Si no existe, devuelve `null` (no `undefined`).
-

`cancel(id: string): Promise<boolean>`

- Cambia el `estado` de la reserva a `CANCELADA`.
 - Devuelve:
 - `true` si la reserva existía y se actualizó,
 - `false` si no existía (no se modifica nada).
-

`upsertMany(reservas: Reserva[]): Promise<void>`

- Inserta **muchas** reservas de una vez.
 - Debe **evitar duplicados por id**:
 - Si ya existe una reserva con ese `id`, **no la insertes de nuevo**.
 - Recomendación técnica: usar `INSERT ... ON CONFLICT(id) DO NOTHING` o lógica equivalente en transacción.
-

Reglas obligatorias

- No debe haber lógica de negocio aquí (solo persistencia).
 - No debe modificar las reservas recibidas (no mutar los objetos).
 - El repositorio debe funcionar aunque la BD esté vacía (tabla recién creada).
-

Ejercicio 4 – REST (1,5)

 Archivo: `src/rest/ReservaRestService.ts`

Debes implementar un **cliente REST** encargado únicamente de la comunicación con una API de reservas.

Métodos a implementar

- `getReservas(): Promise<Reserva[]>`
Realiza una petición `GET` al endpoint de reservas y devuelve el listado completo.
- `getReserva(id: string): Promise<Reserva>`
Realiza una petición `GET` al endpoint de una reserva concreta usando su `id`.
- `createReserva(reserva: Reserva): Promise<Reserva>`
Realiza una petición `POST` enviando la reserva en el cuerpo como JSON.
Devuelve la reserva creada.
- `updateReserva(id: string, patch: Partial<Reserva>): Promise<Reserva>`
Realiza una petición `PUT` o `PATCH` al endpoint de la reserva indicada.
Envía únicamente los campos a modificar y devuelve la reserva actualizada.
- `deleteReserva(id: string): Promise<void>`
Realiza una petición `DELETE` al endpoint de la reserva indicada.
No devuelve contenido.

Reglas obligatorias

- Todas las peticiones deben realizarse usando `fetch`.
- En las peticiones con cuerpo (`POST`, `PUT` o `PATCH`) debes usar:
 - `Content-Type: application/json`
- Si `response.ok === false`, debes **lanzar un Error**.

- No captures ni silencies errores: deben propagarse.

Ejercicio 5 – Servicio (1)

 Archivo: `src/services/ReservaService.ts`

Debes implementar una **capa de servicio** que permita trabajar de forma transparente con distintos orígenes de datos.

Selección del origen

- **LOCAL** → repositorio SQLite.
- **REMOTO** → cliente REST.

El resto de la aplicación **no debe conocer** el origen real de los datos.

Métodos a implementar

- `init(): Promise<void>`
Inicializa el origen local si es necesario.
- `listar(): Promise<Reserva[]>`
Devuelve todas las reservas desde el origen configurado.
- `obtener(id: string): Promise<Reserva | null>`
Devuelve la reserva indicada o `null` si no existe.
- `crear(reserva: Reserva): Promise<void>`
Crea una nueva reserva en el origen correspondiente.
- `cancelar(id: string): Promise<boolean>`
Cancela la reserva indicada.
Devuelve `true` si la reserva existía, `false` en caso contrario.

Ejercicio 6 – Sincronización (1)

 Archivo: `src/services/sync.ts`

```
syncRemotoALocal(  
  repo: ReservaRepository,  
  rest: ReservaRestService  
): Promise<{ descargadas: number; insertadas: number }>
```

Reglas

- Descarga todas las reservas desde el sistema remoto.
- Inserta las reservas en la base de datos local.
- No deben crearse duplicados (identificados por el `id` de la reserva).
- Devuelve un objeto con:
 - `descargadas`: número total de reservas obtenidas del remoto.
 - `insertadas`: número de reservas nuevas insertadas en local.

Cómo ejecutar (genera nota automáticamente)

```
npm install  
npm test
```

Al finalizar se crean en la raíz:

- `nota.json` (detalle por ejercicio)
- `nota.txt` (resumen)

Importante: no ejecutes `./node_modules/.bin/jest` directamente. Usa `npm test`.

Nota 2: Si quisieras ejecutar sólo una parte de los test:

```
npx jest tests/domainModels.test.ts
```



Salida de nota

- `nota.json` incluye desglose por ejercicio y nota final /10
- `nota.txt` es un resumen rápido